开放原子开源基金会
OPENATOM FOUNDATION

OpenHarmony

2026 开源鸿蒙大学生创新大赛 - 赛道一

HapTest

面向 OpenHarmony 应用的多模态智能测试系统

OpenHarmony 应用的智能测试与性能观测基础设施

多模态观察 · 智能测试 · 性能观测

北京航空航天大学 · SMAT 团队

刘发荣 · 2026 年 11 月

基于 FSE'25 / ASE'25 研究积累与 2026 系统级实现

CONTENTS

目录

五个部分

01 背景与问题

生态在长大，测试仍在用单点脚本追赶

02 方案与创新

五模块闭环、 PTG 、设备 API 、 CLI 、性能观测

03 系统实现

四层架构、关键模块、工程规范

04 验证与成果

FSE'25 、 ASE'25 、当前仓库可复现证据

05 价值与展望

应用场景、路线图、团队与研究积累

OpenHarmony 应用的多模态智能测试，缺的不是工具，而是闭环

系统在长大，测试仍在用单点脚本追赶；多模态观察、状态、执行、证据彼此割裂

15,000+
OpenHarmony 应用（公开研究数据）

9 亿
覆盖设备（公开研究数据）

26
20 款商业应用中发现的问题（ FSE'25 ）

人工测试

依赖业务经验，能判断页面含义；
但重复操作耗时，操作路径难以完整保留。

例：登录态、弹窗与返回路径靠人记

脚本测试

稳定、可回归；
但定位器、等待条件与初始化逻辑需要持续维护，页面一改就失效。

例：页面改版后定位器批量失效

随机 / 规则探索

能自动产生操作；
但难以表达业务目标，也难以把一次探索沉淀为可复现资产。

例：能点遍页面，说不清业务是否完成

缺口：没有一个层把“观察 → 状态 → 动作 → 证据”串起来

多模态观察是入口：控件树 + 截图 + 日志 + 任务语义；HapTest 补的正是这层编排。

多模态智能测试：把测试意图变成可执行、可复现、可观测的证据

五模块闭环，共享同一份多模态观察、同一张页面图、同一套执行器

01

多模态观察

截图、控件树、日志与任务语义，生成带 observationId 的多模态观察记录。

产出： observationId + 树 + 截图

02

状态理解

解析页面与控件，构建 PTG 页面迁移图；保留业务字段差异。

产出： PTG 节点与迁移边

03

路径规划

候选生成、过滤、评分与选择；随机、 DFS 、静态、 LLM 可替换。

产出：下一步动作 a(t)

04

受约束执行

观察先行、过期即拒、歧义不猜、设备独占；执行后立即失效 observation 。

产出：新观察 O(t+1)

05

证据沉淀

日志、截图、 PTG 、检查点与失败现场；回放策略支持回归。

产出：用例 / 报告 / 证据

研究底座

FSE'25 模块化动态分析框架

系统级新增

2026 ：设备 API / CLI / 会话 / E2E

研究延伸

ASE'25 遮挡诊断 / 性能观测

分层架构：设备差异下沉，策略差异上浮

四层 + 一个贯穿始终的 observation / 证据模型；同一套 API 服务 CLI、Fuzz、Web 与未来 Agent

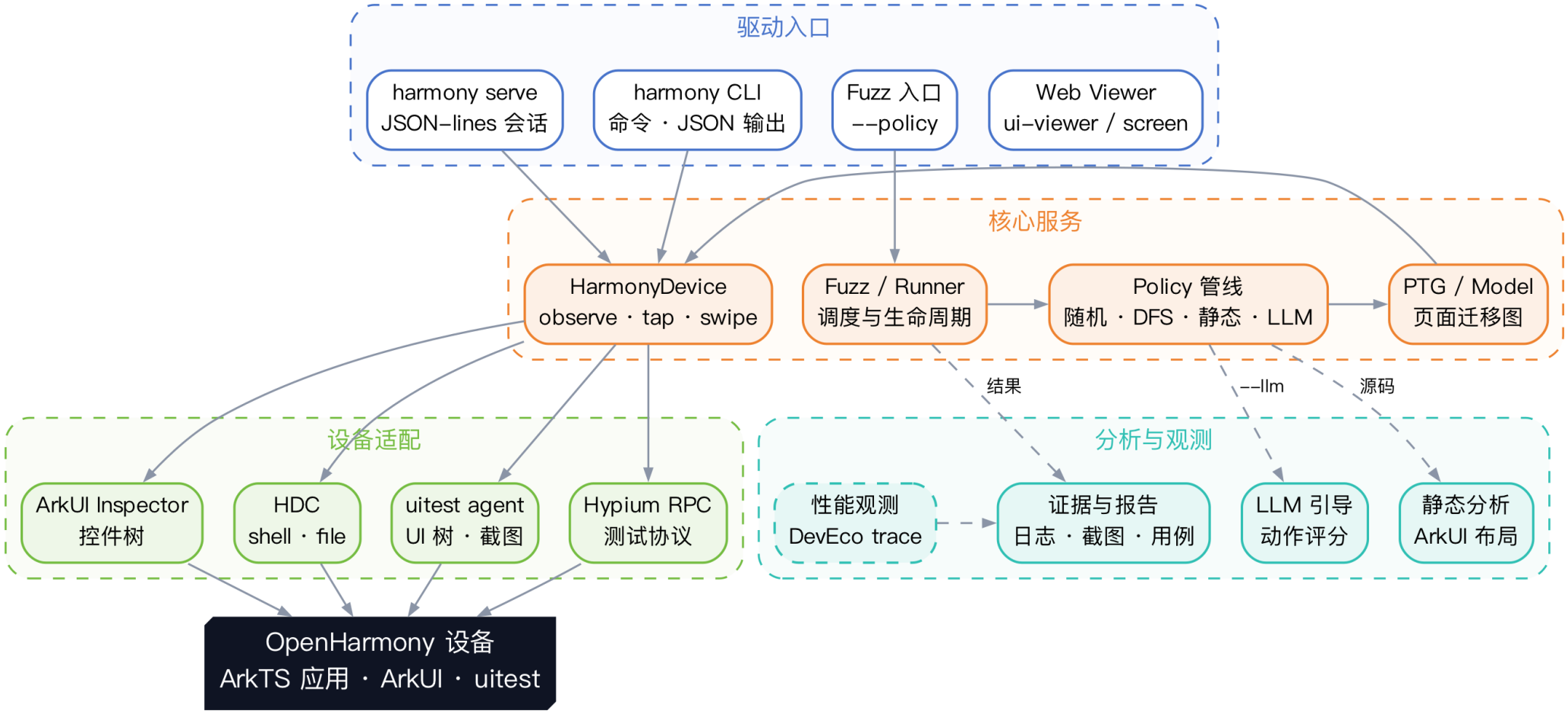
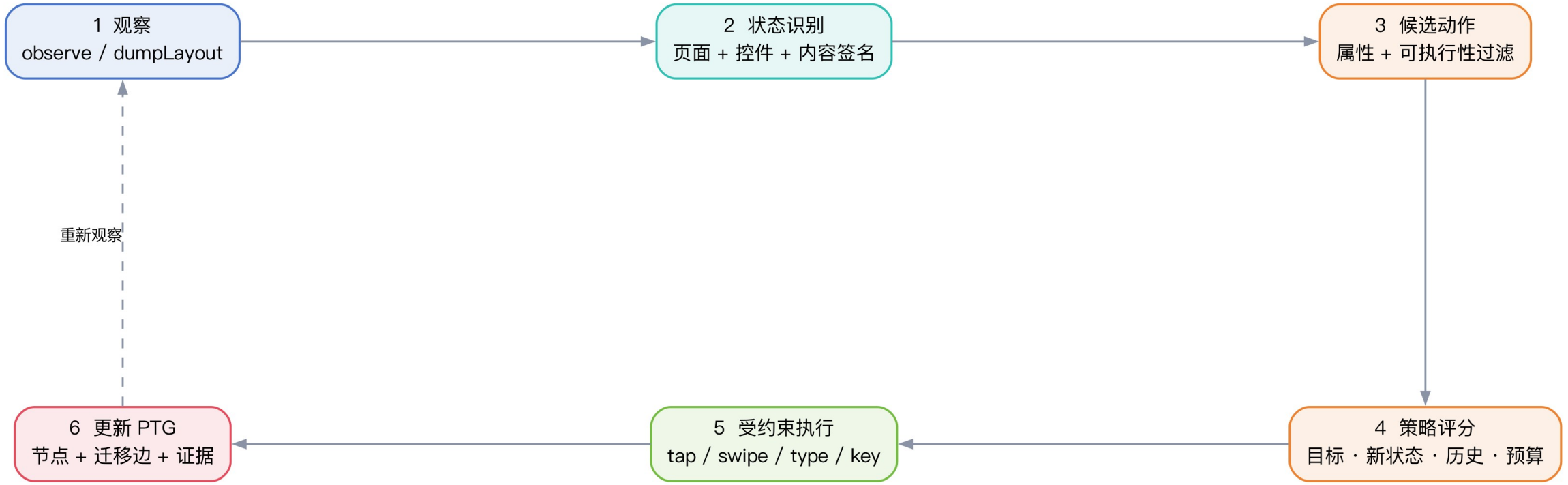


图 1 HapTest 系统总体架构（实现与设计分层标注）

创新 1 ： PTG 把智能测试过程变成可回溯的状态图

每一条边都来自真实执行后的观察，而不是模型猜测；状态图同时约束重复与预算



状态图

页面节点 + 迁移边 + 执行证据；支持可达性、路径回溯与 SVG 导出。

策略管线

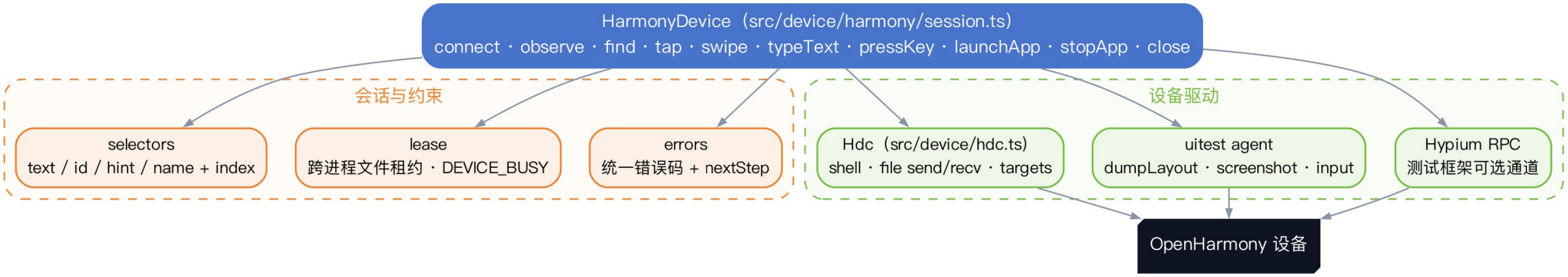
候选 → 过滤 → 评分 → 选择；随机 / DFS / 静态 / LLM 可替换。

复用

同一轨迹进入回放与回归；失败证据与页面变化一起保存。

创新 2 ：让“不猜”成为设备层的系统行为

observationId 、过期检测、歧义拒绝、跨进程租约、锁屏与敏感字段保护，全部由 HarmonyDevice 强制



安全不变量（由 HarmonyDevice 强制）

① 动作前必须有 observation；mutate 后立即失效

② 坐标动作绑定 observationId + bundle + 屏幕尺寸 + contentSig

③ 任一变化 → STALE_OBSERVATION，不执行动作

④ selector 多候选 → AMBIGUOUS_TARGET + 候选列表，不猜

⑤ 动作异常 → ACTION_OUTCOME_UNCERTAIN，replayAction=false

⑥ 同一 serial 文件租约独占；锁屏不返回树与截图

⑦ 认证字段命中敏感词 → SENSITIVE_FIELD，拒绝代输

观察先行

动作前必须有 observation ； mutate 后立即失效。没有 observation 的动作直接拒绝。

失败关闭

歧义返回候选、过期返回 STALE 、异常标记 ACTION_OUTCOME_UNCERTAIN ；不猜、不重放。

设备独占

同一 serial 跨进程文件租约；锁屏不返回树与截图；认证字段拒绝代输。

创新 3 ： CLI + JSON-lines ，让 Agent 与 CI 用同一套语义

stdout 永远是 JSON ；退出码稳定；失败带 errorCode 与 nextStep

haptest harmony · 真实命令与输出格式

```
$ haptest harmony observe --screenshot before.png
{"ok":true,"observation":{"observationId":"o-7f3a",...}}

$ haptest harmony tap --text "搜索" --index 0
{"ok":true,"result":{"outcome":"completed"}}

$ haptest harmony swipe --from 500,1800 --to 500,600
{"ok":true,"result":{"outcome":"completed"}}

$ haptest harmony doctor
{"ok":false,"errorCode":"DEVICE_NOT_READY",...}
nextStep: "Connect and authorize one device..."

$ haptest harmony serve
{"id":1,"method":"observe","params":{}}
{"id":1,"ok":true,"result":{"observationId":"o1"}}
```

一次性命令

doctor / devices / observe / tap / type / key / swipe / launch / stop ；先 observe 再动作。

有状态会话

serve 每行一个请求、每行一个响应； observationId 可跨多步复用。

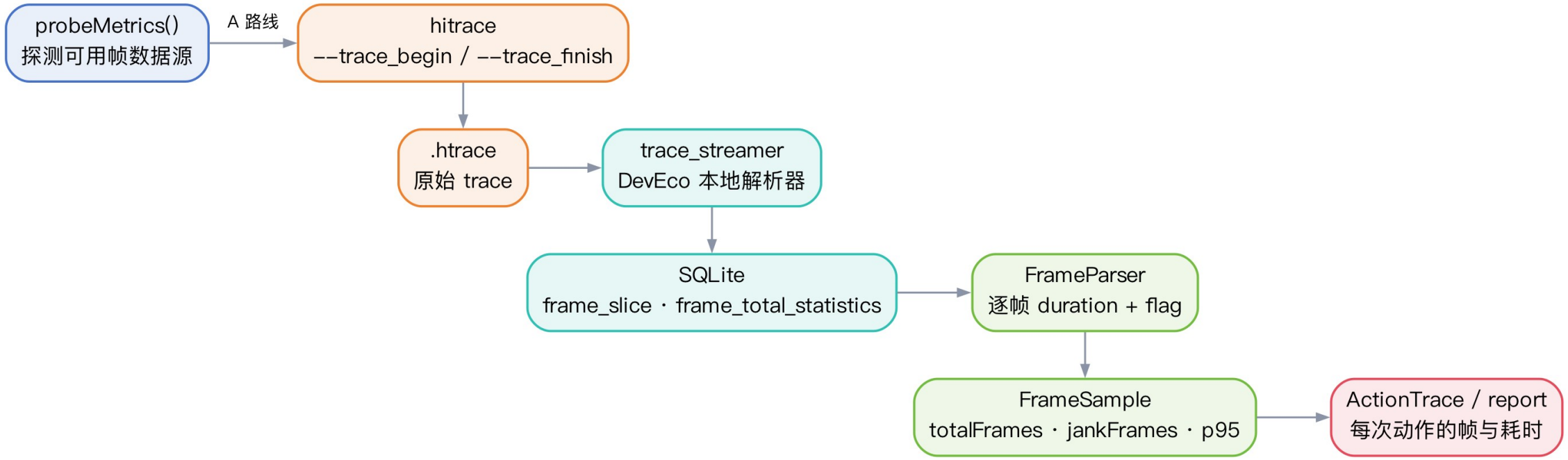
退出码

0 成功 / 1 动作失败 / 2 用法错误 / 3 前置条件不满足；错误面可被脚本判断。

同一套 API 语义 → CLI / serve / 未来 Agent 宿主；不需要为每个宿主重写一层设备适配。

创新 4 ：性能观测，从“ fps 采样”转向“逐帧切片”

hitrace → .htrace → trace_streamer → SQLite frame_slice → FrameSample ；缺数据就明确报错



为什么不用 fps

fps 采样回答不了“哪一帧、哪个动作、哪段时间”；逐帧切片才能把性能与步骤对齐。

probe / trace

probeMetrics 探测来源； trace(action, run) 用同进程包住动作窗口； FrameSample 输出帧数与 p95 。

失败关闭

来源不可用 → UNSUPPORTED_METRIC ；不返回 fps:0 ，不编造帧数；报告显式列出缺失。

工程实现：可构建、可测试、可复现

TypeScript · vitest · 真机 E2E · GitHub Actions ；提交前 build 与 test 必须全绿

74
src/ TypeScript 文件

10,843
src/ 代码行数

60
单元测试（ 20 个文件 ）

37
本次分支变更文件（ +6,794 ）

构建与测试记录 · 2026-11-07

```
$ npm run build
> tsc   ( exit 0 )

$ npm test -- --run
Test Files  20 passed (20)
Tests       60 passed (60)
Duration    3.89s

$ npm run e2e
SKIPPED: set HAPTEST_E2E_SERIAL ...
（无设备时不伪造结果，退出 0 ）
```

代码结构

src/cli · device · model · policy · runner · ui ； lib/ 为构建产物。

测试标准

只写会失败的测试；真实输出、真实 IO 、状态机韧性、安全不变量。

CI

GitHub Actions ： Linux / Windows ， Node 18 / 20 / 21 。

文档

设计规格、设备 API 手册、性能逆向记录、 AGENTS.md 提交规范。

真实应用验证： 20 款商业应用， 26 个未报告问题

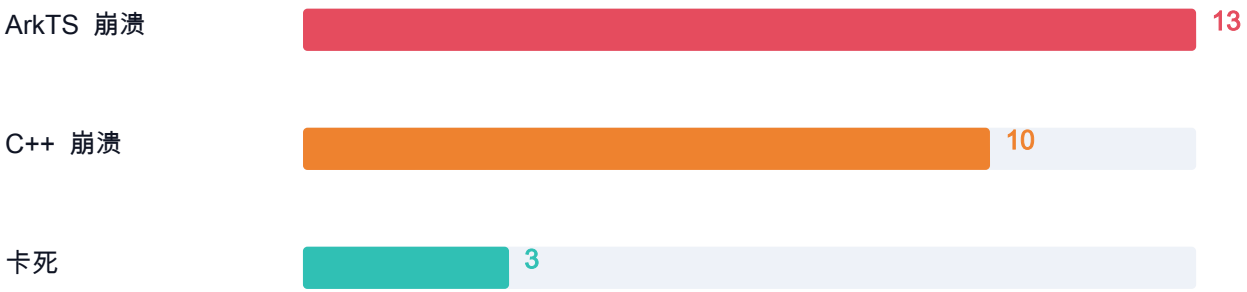
FSE 2025 Industry Track · HUAWEI Mate60 / OpenHarmony 5.0.0 · 贪心 DFS + 故障日志

20
参与实验的商业应用

11
发现问题的应用

26
此前未报告的运行问题

问题类型分布（项）



13 ArkTS 崩溃 · 10 C++ 崩溃 · 3 卡死；问题分布在 11 / 20 款应用。

来源标注：以上数据来自 FSE'25 论文表 4；本仓库为方法实现与工程化，不将论文实验重复声称为本次新增。


HapTest: The Dynamic Analysis Framework for OpenHarmony

Farong Liu[1] Beihang University China
Mingyi Zhou[1] Beihang University China
Yukun Zhang Peking University China
Ting Su East China Normal University China
Bo Sun Huawei China
Jacques Klein University of Luxembourg Luxembourg
Xiang Gao Beihang University China
Li Li[2] Beihang University China

FSE 2025

Industry Track

HapTest: The Dynamic Analysis Framework for OpenHarmony

第一作者：刘发荣
北京航空航天大学

多模态遮挡诊断： 100 款应用、 3326 万个重叠实例

ASE 2025 Industry Track · HCO-Eye / HCOBench · 精确率 97.14% / 召回率 89.47% / F1 93.12%

100
分析的热门应用

33.26M
UI 重叠实例

34
高开销遮挡案例

93.12%
HCOBench F1

HCOBench 指标（ % ）



HCOBench ： 152 个挑战样本、 38 个人工确认正例；正确检出 34 个高开销遮挡案例。


An Empirical Study on UI Overlap in OpenHarmony Applications

Fatong Liu, Mingyi Zhou*, Li Li*,
School of Software
Beihang University, Beijing, China

Abstract—UI overlap is a phenomenon where one UI component visually covers another. While this overlap is necessary to construct rich visual hierarchies, it is also a root cause of usability issues and performance bottlenecks. However, a systematic, data-driven understanding of UI overlap in real-world apps is largely lacking. To bridge this gap, we conduct the first large-scale empirical study on UI overlap in the OpenHarmony ecosystem. We analyze 100 popular apps, classifying 33,262,624 overlap instances through a novel clustering algorithm. Our findings reveal that overlap is a deliberate design practice rather than a defect. However, we identify a critical, hard-to-detect performance defect where resource-intensive components are rendered while visually obscured. We propose HCO-Eye, a multimodal vision-language model (VLM)-based approach, to automatically detect such issues, successfully identifying 34 high-cost occlusion cases in commercial apps. Our study not only provides the first comprehensive understanding of UI overlap in OpenHarmony but also offers a practical tool to automatically diagnose complex performance-related UI bugs. Our tools are publicly available.

I. Introduction

The Graphical User Interface (GUI) serves as the visual bridge connecting users and software applications. With the rise of the OpenHarmony ecosystem [1], [2] and the popularization of declarative UI frameworks like ArkUI, the development paradigm for mobile applications is undergoing a profound transformation, leading to a significant increase in the complexity of UI hierarchies and interaction patterns. As user expectations for richer, more interactive, and visually appealing experiences grow, developers are increasingly and willingly adopting complex design patterns. This shift is driven by the need to deliver sophisticated functionality and dense information within the constrained screen space of mobile devices.

UI overlap [3], [4] is an indispensable "double-edged sword" in modern UI design. On one hand, it is a necessary design technique for implementing modern features such as floating action buttons, stacked cards, and modal dialogs, which are essential for creating rich visual hierarchies and fluid user experiences. On the other hand, it is a major source of problems, leading to usability issues like obscured text and unclickable buttons, as well as performance bottlenecks caused by phenomena such as overdraw and occlusion. Effectively managing and leveraging UI overlap has thus become a core challenge in ensuring application quality and user experience.

Nevertheless, the characterization and detection of UI overlap issues are non-trivial. First, little is known about how prevalent overlap is in the OpenHarmony ecosystem and what the common patterns are. Second, existing UI defect detection tools are often platform-specific, typically targeting major ecosystems like Android and iOS, and cannot be directly applied to OpenHarmony. Third, and most importantly, traditional static [5], [6] or dynamic [7] analysis tools lack the semantic understanding to distinguish between intentional design and unintentional defects, let alone identify complex performance issues like high-cost occlusion. This paper defines high-cost occlusion as a scenario where a high-cost component is rendered despite being fully or partially covered by other UI elements, thereby wasting valuable computational resources.

For the purpose of this study, we define a *high-cost component* based on the following criteria:

- **Heavy Rendering Type:** Components that are inherently expensive to render, such as a Video Player, Complex Animation, Map, Camera Preview, Long Image List, or WebView.
- **Heavy Computation/IO Type:** Real-time refreshing components that involve significant background processing, such as Complex Charts or Data-Heavy Lists that update frequently.

To bridge these gaps, this paper conducts the first large-scale empirical study of UI overlap. We choose the OpenHarmony platform as the target. OpenHarmony is a new mobile platform with rapidly evolving applications, which usually have a complex UI structure. This study is guided by the following four core Research Questions (RQs):

- **RQ1: The Prevalence of UI Overlap.** How prevalent is UI overlap in real-world OpenHarmony applications?
- **RQ2: The Classification of UI Overlap.** What are the common, reproducible patterns of UI overlap?
- **RQ3: A Detection Tool for Performance-Related Overlap.** To what extent can we automatically identify performance-related overlap issues?

To answer these questions, we perform an in-depth analysis of 100 popular apps, propose a novel taxonomy to classify overlap patterns, and design an innovative tool, HCO-Eye, which leverages a multimodal vision-language model to automatically detect performance-critical defects. The main contributions of this paper include:

*Corresponding authors

ASE 2025

Industry Track

An Empirical Study on UI Overlap in OpenHarmony Applications

第一作者：刘发荣
北京航空航天大学

来源标注：数据来自 ASE'25 论文； VLM 多模态诊断为研究延伸，未并入当前仓库主链路。

当前仓库的可复现证据

构建、单测、 E2E 、 CLI 输出都可重跑；无设备时不伪造结果

证据	命令 / 文件	结果
构建	npm run build	tsc 退出 0
单元测试	npm test -- --run	20 files / 60 tests passed
真机 E2E	npm run e2e	无 serial 时 SKIPPED ，退出 0
CLI 诊断	haptest harmony doctor	DEVICE_NOT_READY + nextStep
代码规模	src/	74 files / 10,843 lines
本次变更	git diff main...HEAD	37 files / +6,794 / -23

证据链

1 **观察**
observationId + 树 + 截图

2 **动作**
selector / 坐标 + 约束

3 **PTG**
页面节点 + 迁移边

4 **报告**
日志 + 截图 + 失败现场

可复现 · 可追溯 · 不造假

验证原则：不把 mock 当真实；无设备时明确 SKIPPED ；每个数字有来源；论文与专利按来源标注。

从一次多模态智能测试到可复用的质量资产

开发者、测试团队与生态伙伴复用同一套观察、执行与证据

应用开发者

新增 / 修改页面后，用自然语言提出验证目标；
获得可执行步骤、检查点与失败证据；
有效轨迹沉淀为回归用例。

典型：新增页面后的保存 / 返回验收

测试团队

缺陷复现、稳定性压测与探索性测试；
统一定位器、等待条件与证据格式；
减少重复操作与用例维护。

典型：崩溃复现与版本回归

生态伙伴

面向 OpenHarmony 开发者的测试基础设施；
可接入 CI 、教学与工具链；
方法、代码与文档可复用。

典型： CI 冒烟与证据归档

路线图

现在

设备 API / CLI / 会话 / E2E

已实现并有测试覆盖

下一步

帧解析 / 性能报告 / 多设备

真机日补齐枚举值，扩展采集

后续

CI 集成 / Agent 宿主 / 应用试点

把证据链接入研发流程

团队与研究积累

北京航空航天大学 SMAT 团队 · 刘发荣 · 全栈交付

刘

刘发荣

队长 · 架构 / 设备层 / CLI / 测试 / 文档

北京航空航天大学 · SMAT 团队

覆盖需求、架构、实现、测试、文档与演示；
论文与专利为研究积累，按来源标注。

研究： FSE'25 / ASE'25 / 发明专利
开源： SMAT-Lab / HapTest （ Apache-2.0 ）

作品仓库： github.com/growvv/HapTest-2026
GitCode ： gitcode.com/rogn/HapTest-2026


HapTest: The Dynamic Analysis Framework for OpenHarmony

FSE 2025

HapTest 动态分析框架
第一作者


An Empirical Study on UI Overlap in OpenHarmony Applications

ASE 2025

UI 重叠实证研究
第一作者


国家知识产权局

发明专利

OpenHarmony 动态分析框架
申请号 202510375959.1



开源项目

SMAT-Lab / HapTest
Apache-2.0

工作流

架构设计 · 设备适配 · CLI/ 会话 · 策略与模型 · 测试 /E2E · 文档 · PPT · 视频

让测试意图变成可执行、可复现、可观测的证据。

谢 谢



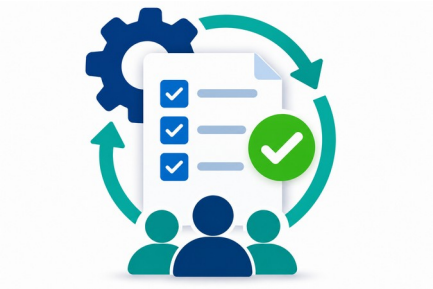
性能观察

看清每一次卡顿



多设备支持

覆盖更多终端



日常自动检查

融入团队流程

HapTest · 面向 OpenHarmony 应用的多模态智能测试系统

北京航空航天大学 SMAT 团队 · 刘发荣 · 2026 年 11 月

作品仓库： github.com/growvv/HapTest-2026 · GitCode ： gitcode.com/rogn/HapTest-2026